Домашнее задание по курсу «Введение в архитектуру вычислительных систем»

ALU Register — вариант 3

Шелонин Арсений Б01-411

shelonin.ak@phystech.edu

Содержание

1 Запуск симуляции с BHT **2**

2 Снижение точности до 0% **4**

3 Бимодальный предсказатель **5**

4 Обман бимодального предсказателя **6**

1 Запуск симуляции с BHT

Исходный код на ассемблере (RISC-V)

bpu_v1.asm:

```
.text
main:
    li      t0, 10000
    li      a3, 1
    li      a4, 0
    li      a1, 0
loop:
    beq     a4, zero, magic_br_1 # branch #1
    addi    a4, a4, 0
magic_br_1:
    beq     a3, zero, magic_br_2 # branch #2
    addi    a3, a3, 0
magic_br_2:
    addi    a1, a1, 1
    bne     a1, t0, loop         # branch #3

```

Параметры BHT

Number of entrie	8
History size	1
Initial value	NOT TAKEN

Запустив симуляцию, получим такое конечное состояние таблицы:

Для определения, какое ветвление соответствует какому слоту таблицы, сбросим таблицу к незаполненному состоянию и запустим симуляцию заново пошагово.

Идя пошагово дальше, определим индексы в таблице для каждого ветвления и по рис. 1 определим его точность.

branch	BHT index	accuracy
`branch 1`	5	99.99%
`branch 2`	7	100.00%
`branch 3`	2	99.98%

Branch History Table Simulator

# of BHT entries 8 **BHT history size** 1 **Initial value** NOT TAKE

Instruction: bne x11,x5,-20

@ Address: 0x400028

-> Index: 2

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0.00
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	9998	2	99.98
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	T	TAKE	9999	1	99.99
6	NT	NOT TAKE	0	0	0.00
7	NT	NOT TAKE	10000	0	100.00

Рис. 1: Состояние таблицы в конце симуляции

Branch History Table Simulator

# of BHT entries 8 **BHT history size** 1 **Initial value** NOT TAKE

Instruction: beq x14,x0,8

@ Address: 0x400014

-> Index: 5

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0.00
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	0	0	0.00
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	NT	NOT TAKE	0	0	0.00
6	NT	NOT TAKE	0	0	0.00
7	NT	NOT TAKE	0	0	0.00

Рис. 2: Состояние таблицы при встрече первого ветвления

- `branch 2`: условие не выполнится никогда, так как там идёт сравнение постоянного не равного нулю регистра `a3` с нулём, а так как изначальное значение таблицы установлено в `NOT TAKEN`, точность предсказания этого перехода 100%.

- `branch 1`: полностью аналогично, только условие выполняется всегда. Изначальное значение таблицы установлено в `NOT TAKEN` поэтому `BHT miss` произойдёт только в первый раз.

- `branch 3`: 2 промаха: в первый раз — когда заходим в цикл, а таблица предсказывает `NOT TAKEN`, и в последний — при выходе из цикла, а таблица предсказывает `TAKEN`.

2 Снижение точности до 0%

Сейчас используется BHT с историей длиной 1, то есть предсказатель описывается следующим конечным автоматом состояний:

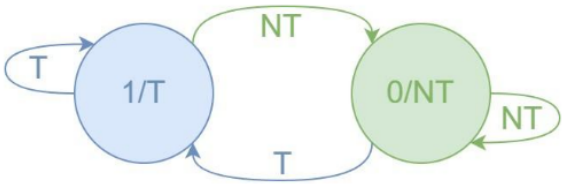


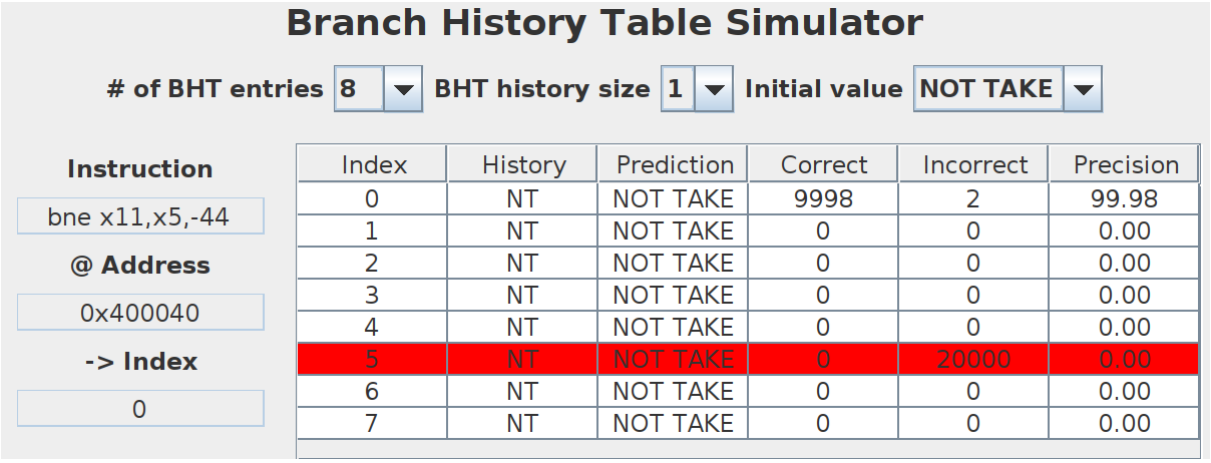
Рис. 3: Конечный автомат предсказателя с историей длиной 1

Тогда, чтобы снизить точность предсказания до 0% нужно, чтобы бранчи попали в одну ячейку `BHT` и результат условий чередовался. Индекс в таблице рассчитывается как

$$\texttt{index} = (\texttt{address} \gg 2) \bmod 8$$

Адрес инструкций кодируются через 0x4, поэтому для того, чтобы не учитывать 2 не значимых нуля в битовом представлении, делаем логический сдвиг вправо на 2.

Тогда, поставив 6 инструкций `nop` между бранчами, добьемся того, что они будут попадать в одну ячейку таблицы, а поскольку первый переход всегда выполняется, а второй — нет, предсказание всегда будет неверно:



Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	9998	2	99.98
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	0	0	0.00
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	NT	NOT TAKE	0	20000	0.00
6	NT	NOT TAKE	0	0	0.00
7	NT	NOT TAKE	0	0	0.00

Рис. 4: Состояние таблицы в случае нулевой точности предсказаний

В 1 ячейке таблицы лежит третий переход в тело цикла.

Тестируемый код: task2.asm.

3 Бимодальный предсказатель

Изменим размер истории предсказателя до 2.

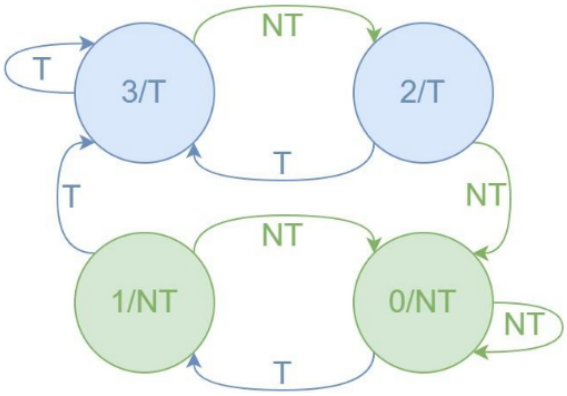


Рис. 5: Конечный автомат предсказателя с историей длиной 2

Теперь состояние предсказателя имеет инертность, и чередующийся результат условия не будет менять предсказываемое значение. Тогда предсказатель будет угадывать в половине случаев:

Branch History Table Simulator

# of BHT entries 8 **BHT history size** 2 **Initial value** NOT TAKE

Instruction

bne x11,x5,-44

@ Address

0x400040

-> Index

0

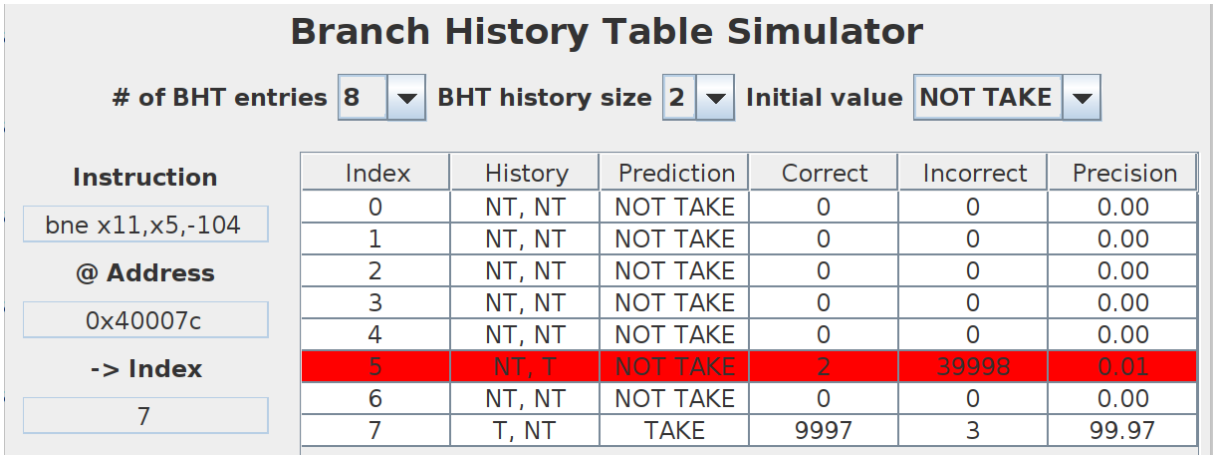
Index	History	Prediction	Correct	Incorrect	Precision
0	T, NT	TAKE	9997	3	99.97
1	NT, NT	NOT TAKE	0	0	0.00
2	NT, NT	NOT TAKE	0	0	0.00
3	NT, NT	NOT TAKE	0	0	0.00
4	NT, NT	NOT TAKE	0	0	0.00
5	T, NT	NOT TAKE	10000	10000	50.00
6	NT, NT	NOT TAKE	0	0	0.00
7	NT, NT	NOT TAKE	0	0	0.00

Рис. 6: Состояние таблицы с расширенной до 2 историей

В первой ячейке находится `branch 3`, перекидывающий `pc` в начало цикла. В первые 2 итерации предсказатель не угадывает, так как начальное состояние — `NOT TAKEN` и предсказатель имеет инертность в 2 итерации, а условие перехода выполняется до самого конца цикла (где предсказатель не угадывает в третий раз).

4 Обман бимодального предсказателя

Добавим `nop` и 2 инструкции условного перехода в блоке после `magic_br_2`, чтобы все 4 ветвления (magic_br_1, magic_br_2 + 2 новых инструкции) не предсказывались. Для этого с помощью `nop` разместитм их все в одной ячейке таблицы. Затем нам нужно построить цикл так, чтобы конечный автомат шел по циклу 1/NT -> 3/T -> 2/T -> 0/NT -> 1/NT. Первая итерация цикла будет устанавливать начальное состояние в 1/NT (изначально оно 0/NT), поэтому на первой итерации предсказатель угадает некоторые переходы.



Branch History Table Simulator

# of BHT entries 8 **BHT history size** 2 **Initial value** NOT TAKE

Instruction: bne x11,x5,-104

@ Address: 0x40007c

-> Index: 7

Index	History	Prediction	Correct	Incorrect	Precision
0	NT, NT	NOT TAKE	0	0	0.00
1	NT, NT	NOT TAKE	0	0	0.00
2	NT, NT	NOT TAKE	0	0	0.00
3	NT, NT	NOT TAKE	0	0	0.00
4	NT, NT	NOT TAKE	0	0	0.00
5	NT, T	NOT TAKE	2	39998	0.01
6	NT, NT	NOT TAKE	0	0	0.00
7	T, NT	TAKE	9997	3	99.97

Рис. 7: Состояние таблицы в случае обмана бимодального предсказателя

Точность переходов $\frac{2}{39998} = 0.00005 < 0.0001$ - условие выполнено.

no_predict2.asm:

```
.text
main:
    li      t0, 10000
    li      a3, 1
    li      a4, 0
    li      a1, 0

loop:
    beq     a4, zero, magic_br_1 # branch #1, 1/NT -> 3/T
    addi    a4, a4, 0

    nop
    nop
    nop
    nop
```

```
    nop
    nop

magic_br_1:
    beq     a3, zero, magic_br_2 # branch #2, 3/T -> 2/T
    addi    a3, a3, 0

magic_br_2:

# ****** ADD HERE ******
# your code for task 4
    nop
    nop
    nop
    nop
    nop
    nop
    beq     a3, zero, lol        # 2/T -> 0/NT
lol:

    nop
    nop
    nop
    nop
    nop
    nop
    nop
    beq     a4, zero, kek         # 0/NT -> 1/NT
kek:
# *********************

    addi    a1, a1, 1
    bne     a1, t0, loop          # branch #3

```